

Experiment 4: Recursion concept

Objective: To understand the concept of **recursion**, its working mechanism, advantages, disadvantages, and practical application areas by implementing a simple recursive function.

Introduction: Recursion is a programming technique in which a function **calls itself** to solve a problem by breaking it into smaller or simpler subproblems.

A recursive function consists of two important components:

- **Base Case:** The condition that stops further recursive calls.
- **Recursive Case:** The part of the function where the function calls itself with a smaller or simpler input.

- ✓ **Recursion = Solve a smaller version of the same problem until a base condition is reached.**
- ✓ Recursion is particularly powerful when the problem itself has a **hierarchical, nested, divide-and-conquer, or backtracking** structure.



Example:

Simple code

```
void printNumbers(int n)
{
    if (n == 0)           // Base case
        return;

    printf("%d ", n);
    printNumbers(n - 1); // Recursive call
}
```

Work process

For $n = 5$, the function works as:

$$5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

↑
Base Case

Advantages of Recursion

Advantage	Explanation
1. Simple code	Complex problems can often be expressed using fewer lines of code.
2. Natural problem solving	Problems that have a recursive structure are easier to represent recursively.
3. Useful for trees and graphs	Tree traversal and many graph algorithms naturally use recursion.
4. Divide-and-conquer	Algorithms such as Merge Sort and Quick Sort use recursive decomposition.
5. Backtracking	Useful for solving problems such as N-Queens, Sudoku and maze solving.
6. Easier implementation	Some mathematical problems such as factorial and Fibonacci are naturally expressed recursively.

Disadvantages of Recursion

Disadvantage	Explanation
More memory usage	Every recursive call creates a new stack frame in memory.
Stack overflow	Too many recursive calls can exceed the available call stack.
Function-call overhead	Repeated function calls can make recursion slower than an equivalent iterative solution.
Difficult debugging	Tracing many nested function calls can be challenging for beginners.
Repeated computation	Some recursive algorithms perform the same calculations repeatedly, such as naïve Fibonacci.
Base-case errors	If the stopping condition is missing or incorrect, infinite recursion may occur.

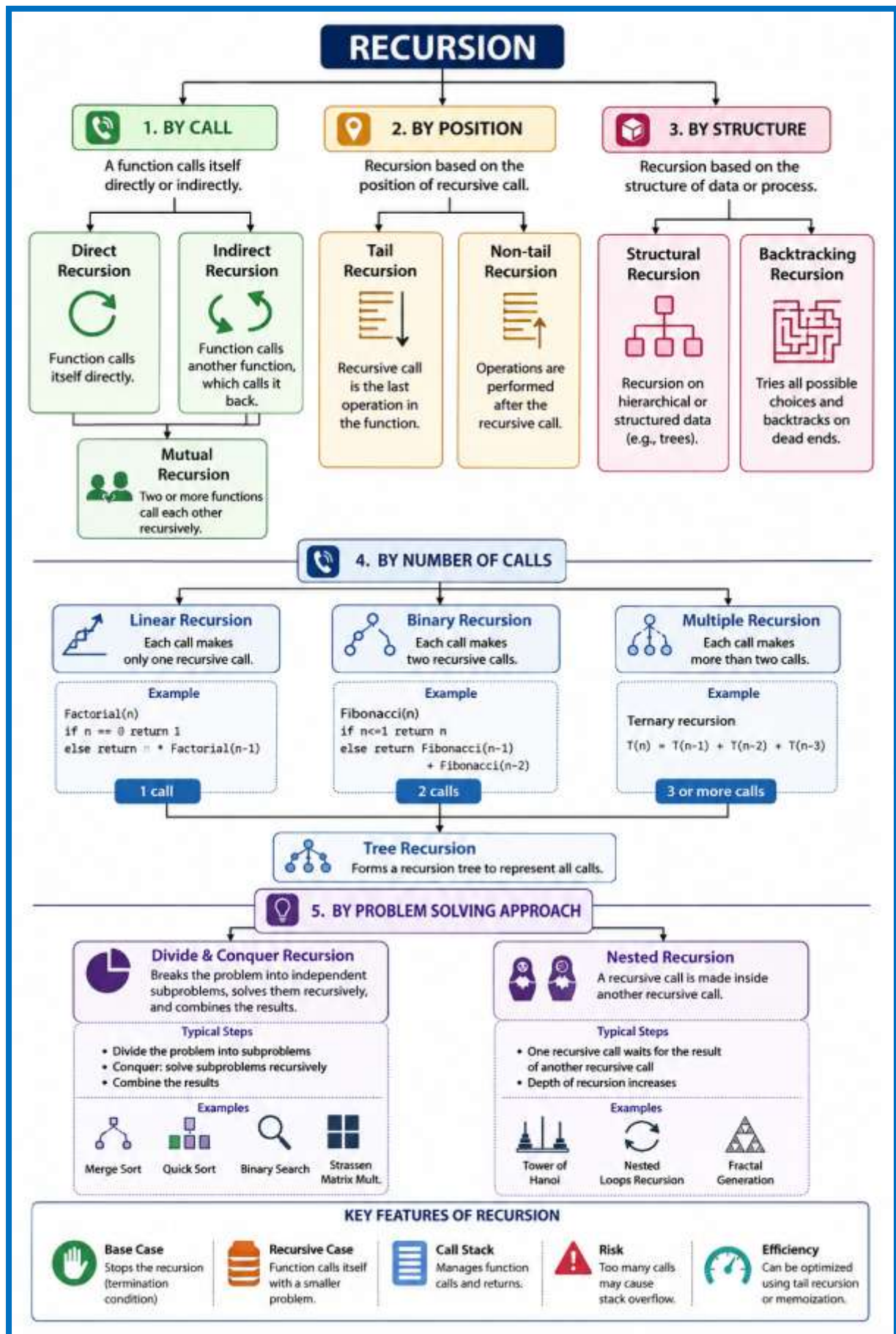
Practical Application Areas

Application Area	How Recursion Is Used	Example
1. File systems	Processes folders and their subfolders	Directory traversal
2. Trees	Visits nodes at different levels	Binary Tree Traversal
3. Graphs	Explores connected nodes	DFS
4. Searching	Divides the search space repeatedly	Binary Search
5. Sorting	Divides data into smaller parts	Merge Sort, Quick Sort
6. AI & Games	Explores possible future states	Game-tree search
7. Backtracking	Tries possible solutions and backtracks	Sudoku, N-Queens
8. Compiler Design	Processes nested structures	Expression/parse trees
9. Web/HTML/XML Processing	Processes nested elements	DOM/tree traversal
10. Mathematical Computation	Solves recursively defined problems	Factorial, Fibonacci, GCD
11. Image Processing	Processes connected regions recursively	Flood-fill
12. Networking	Traverses hierarchical network structures	Network topology/tree traversal

Recursion Types/classification:

S No.	Type	Meaning
1	Direct Recursion	A function directly calls itself
2	Indirect Recursion	Function A calls B, and B eventually calls A
3	Tail Recursion	Recursive call is the last operation
4	Non-Tail Recursion	Some operation remains after the recursive call
5	Head Recursion	Recursive call occurs before processing the remaining work
6	Tree Recursion	A function makes more than one recursive call
7	Nested Recursion	A recursive call is passed as an argument to another recursive call
8	Mutual Recursion	Two or more functions recursively call one another
9	Linear Recursion	At most one recursive call is made at each level
10	Binary Recursion	Exactly/two recursive branches are typically created
11	Multiple Recursion	More than two recursive calls can be made at a level
12	Structural Recursion	Recursion follows the structure of the input data
13	Backtracking Recursion	Recursion explores possibilities and returns to try alternatives
14	Divide-and-Conquer Recursion	Problem is divided into smaller subproblems recursively

Recursion infographic flowchart



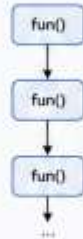
Types of Recursions Visual

TYPES OF RECURSION – OVERVIEW

1 DIRECT RECURSION

A function calls itself directly.

```
void fun(int n) {
    if (n > 0) {
        printf("%d ", n);
        fun(n - 1);
    }
}
```

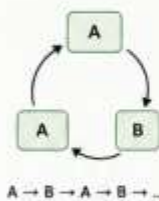


2 INDIRECT / MUTUAL RECURSION

Two functions call each other.

```
void A(int n) {
    if (n > 0)
        B(n - 1);
}

void B(int n) {
    if (n > 0)
        A(n - 1);
}
```



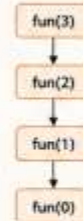
Mutual recursion is essentially a common form of indirect recursion.

3 TAIL RECURSION

The recursive call is the last operation.

```
void fun(int n) {
    if (n == 0)
        return;

    printf("%d ", n);
    fun(n - 1); // last operation
}
```



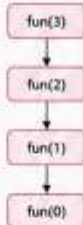
After fun(n-1) returns, there is nothing left to do.

4 NON-TAIL RECURSION

Some operation remains after the recursive call.

```
void fun(int n) {
    if (n == 0)
        return;

    fun(n - 1);
    printf("%d ", n); // after call
}
```



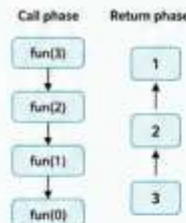
Here printf() happens after the recursive call.

5 HEAD RECURSION

The recursive call occurs before the main processing.

```
void fun(int n) {
    if (n == 0)
        return;

    fun(n - 1);
    printf("%d ", n);
}
```

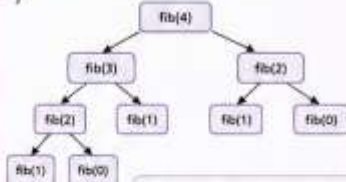


For fun(3): First all calls go down, then values are printed while returning.

6 TREE RECURSION

A function makes multiple recursive calls.

```
int fib(int n) {
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```



This is called tree recursion because the calls form a tree.

7 NESTED RECURSION

A recursive function calls itself inside another invocation of itself.

```
int fun(int n) {
    if (n > 100)
        return n - 10;

    return fun( fun(n + 11) );
}
```



Key idea:
The inner call fun(n + 11) must be evaluated before the outer call.

8 LINEAR RECURSION

Only one recursive call occurs at each level.

```
int factorial(int n) {
    if (n == 0)
        return 1;

    return n * factorial(n - 1);
}
```



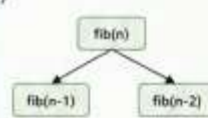
Factorial is both linear recursion and non-tail recursion.

9 BINARY RECURSION

Two recursive calls are made.

```
int fib(int n) {
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}
```



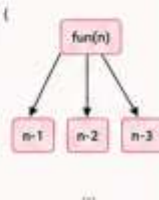
Fibonacci is therefore also a tree recursion.

10 MULTIPLE RECURSION

A function makes more than two recursive calls.

```
void fun(int n) {
    if (n <= 0)
        return;

    fun(n - 1);
    fun(n - 2);
    fun(n - 3);
}
```



This produces multiple branches at every level.

11 STRUCTURAL RECURSION

The recursive call follows the structure of the input.

```
void display(struct Node *p) {
    if (p == NULL)
        return;

    printf("%d ", p->data);
    display(p->next);
}
```



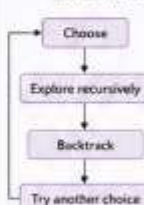
The recursion follows:

Node → next → next → next → NULL

Important for linked lists, trees, and other recursive structures.

12 BACKTRACKING RECURSION

Explore one possibility, return, and then try another.



```
void generate(int n) {
    if (n == 0) {
        printf("Solution\n");
        return;
    }

    generate(n - 1);
    generate(n - 1);
}
```

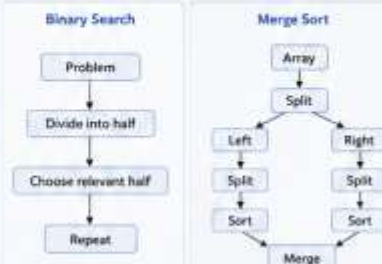
Examples:

- N-Queens
- Sudoku
- Maze solving
- Permutations
- Subsets

Real backtracking: Choose → Explore → Undo → Try another choice

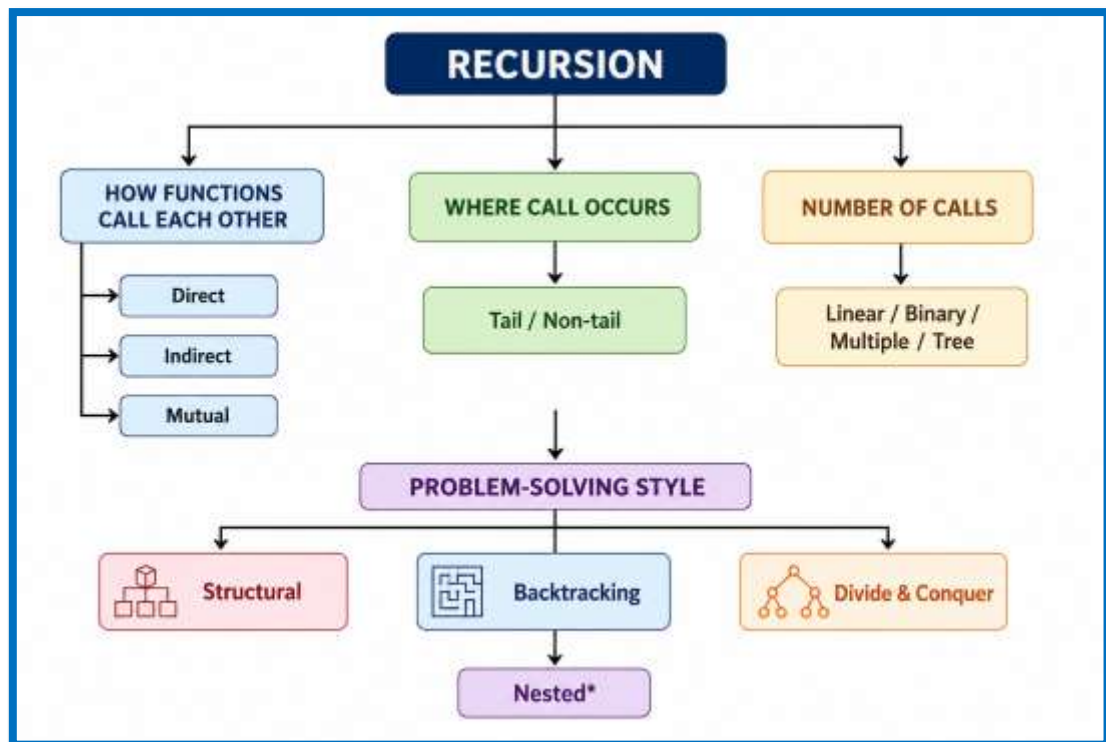
13 DIVIDE-AND-CONQUER RECURSION

The problem is divided into smaller problems.



Examples:

- Binary Search
- Merge Sort
- Quick Sort
- Strassen Matrix Multiplication



Recursion Experiments

Experiment 4.1 — Direct Recursion: Sum of Even Digits

Aim: To write a C program to find the sum of all even digits in a given positive integer using direct recursion.

Concept: Function directly calls itself.

Example: $572846 \rightarrow 2 + 8 + 4 + 6 = 20$

Algorithm

1. Read the number n.
2. If $n == 0$, return 0.
3. Extract the last digit using $n \% 10$.
4. If the digit is even, add it to the recursively calculated sum.
5. Call the function with $n / 10$.
6. Display the sum.

Expected Input: Enter number: 572846

Expected Output: Sum of even digits = 20

Experiment 4.2 — Indirect Recursion: Print Prime Numbers in a Given Range

Aim: To generate prime numbers within a given range using **indirect recursion**.

Concept: One function calls another, which eventually calls the first function.

Function A $\rightarrow$ Function B $\rightarrow$ Function A

Algorithm

1. Read lower and upper limits.
2. Use a function to process each number in the range.
3. Call a second function to recursively check divisibility.
4. The checking function calls the range-processing function when required.
5. Determine whether each number is prime.
6. Display the prime numbers.

Expected Input: Enter range: 10 30

Expected Output: Prime numbers: 11 13 17 19 23 29

Experiment 4.3 — Mutual Recursion: Classify Numbers as Even or Odd

Aim:

To classify a number as even or odd using **mutual recursion**.

Concept: Two functions recursively call each other.

Algorithm

1. Read n.
2. Call even(n).
3. If $n == 0$, return true.
4. Otherwise even() calls odd(n-1).
5. odd() calls even(n-1).
6. Continue until zero is reached.
7. Display Even or Odd.

Expected Input: Enter number: 17

Expected Output: 17 is Odd

Experiment 4.4 — Tail Recursion: Find Maximum Element in an Array

Aim: To find the maximum element in an array using **tail recursion**.

Important: The recursive call is the **last operation**.

Algorithm

1. Read the array and its size.
2. Pass the current index and current maximum to the recursive function.
3. Compare the current element with the maximum.
4. Update maximum if necessary.
5. Recursively process the next element.
6. Stop when all elements are processed.
7. Return the maximum.

Expected Input

Enter size: 6

Enter elements:

12 45 7 89 34 21

Expected Output

Maximum element = 89

Experiment 4.5 — Non-Tail Recursion: Reverse a String

Aim: To reverse a string using **non-tail recursion**.

Concept:

recursive call

↓

more work after return

↓

print character

Algorithm

1. Read the string.
2. Find its length.
3. Recursively process the remaining characters.
4. After the recursive call returns, print the current character.
5. Continue until the complete string is processed.

Expected Input: Enter string: COMPUTER

Expected Output: Reversed string = RETUPMOC

Experiment 4.6 — Head Recursion: Display Array Elements in Reverse Order

Aim: To display array elements in reverse order using **head recursion**.

Concept:

Call recursion first

↓

Return from recursion

↓

Print element

Algorithm

1. Read the array.
2. Start from the first element.
3. Make the recursive call for the next index.
4. Print the current element while returning from recursion.
5. Stop after reaching the last element.

Expected Input: Enter size: 5

Enter elements: 10 20 30 40 50

Expected Output: Reverse order: 50 40 30 20 10

Experiment 4.7 — Nested Recursion

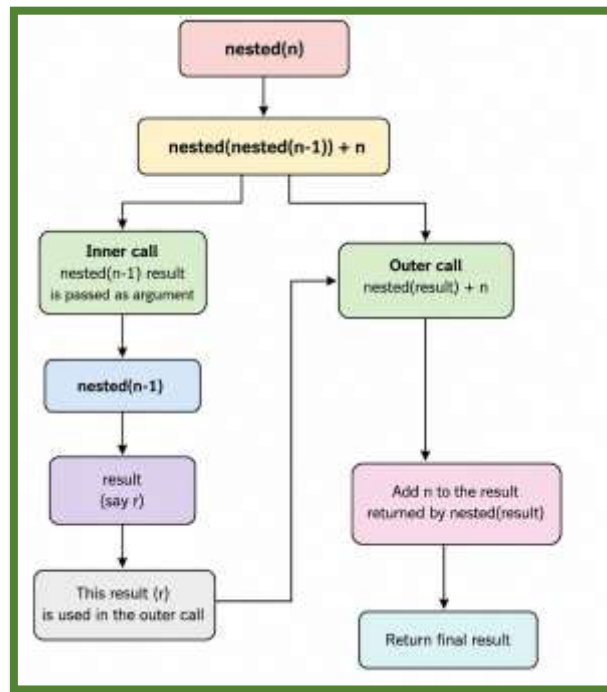
Aim: To write a C program to demonstrate **nested recursion**, where the result of one recursive function call is used as the argument of another recursive call.

Function Definition

$$N(n) = \begin{cases} 1, & n \leq 0 \\ 2, & n = 1 \\ N(N(n-1)) + n, & n > 1 \end{cases}$$

Key Nested Recursion: `nested(nested(n - 1)) + n`

The **inner nested()** call executes first, and its result becomes the input to the **outer nested()** call.



Algorithm

1. **Start.**
2. Read the value of `n`.
3. Call the function `nested(n)`.
4. If `n <= 0`, return 1.
5. If `n == 1`, return 2.
6. For `n > 1`, first calculate `nested(n-1)`.
7. Use the result of this recursive call as the argument of another `nested()` call.
8. Add `n` to the result.
9. Return the final value.
10. Display the result.
11. **Stop.**

Expected Input and Output

Test Case 1

Enter `n`: 2

Result = 4

Test Case 2

Enter `n`: 4

Result = 11

Experiment 4.8: Linear Recursion

Aim: To write a C program to find the sum of elements in an array using linear recursion.

Problem Statement: Given an array of n integers, find the sum of all its elements using recursion. The recursive function should make only one recursive call at each step.

Algorithm

1. Start.
2. Read the number of elements n.
3. Read the n array elements.
4. Call the recursive function sum(a, n).
5. If $n == 0$, return 0.
6. Otherwise, add the last element $a[n-1]$ to $\text{sum}(a, n-1)$.
7. Return the calculated sum.
8. Display the sum.
9. Stop.

Expected Input: Enter number of elements: 5

Enter elements: 10 20 30 40 50

Expected Output: Sum = 150

Recursion Flow

sum(5)
↓
sum(4)
↓
sum(3)
↓
sum(2)
↓
sum(1)
↓
sum(0)

Experiment 4.9 — Binary Recursion: Find Frequency of an Element in an Array

To make this genuinely **binary recursive**, divide the array into two portions and recursively search both portions.

Aim: To find the frequency of a given element in an array using binary recursion.

Algorithm

1. Read the array and search element.
2. Divide the array into two halves.
3. Recursively count occurrences in the left half.
4. Recursively count occurrences in the right half.
5. Add both counts.
6. Return the total frequency.

Expected Input

Array:

2 5 2 8 2 7 5 2

Search element: 2

Expected Output

Frequency of 2 = 4

Experiment 4.10 — Structural Recursion: Problem: Display Linked List Values

Aim:

To display all elements of a linked list using **structural recursion**.

Algorithm

1. Create a linked list.
2. Pass the first node to the recursive function.
3. If the current node is NULL, stop.
4. Display the current node's data.
5. Recursively process node->next.
6. Continue until NULL.

Expected Input

Number of nodes: 5

Elements:

10 25 30 45 60

Expected Output

Linked List:

10 → 25 → 30 → 45 → 60 → NULL

Concept:

Node

↓

next

↓

next

↓

NULL

The recursion follows the **structure of the linked list**.

Very important: One problem can belong to multiple types

Problem	Possible classifications
Factorial	Direct + Linear + Non-tail
Factorial with accumulator	Direct + Linear + Tail
Fibonacci	Direct + Binary + Tree + Non-tail
Merge Sort	Direct + Binary + Tree + Divide-and-Conquer
Quick Sort	Direct + Divide-and-Conquer; often binary/multiple depending on partition treatment
Binary Search	Direct + Linear-call + Divide-and-Conquer + Tail/Non-tail depending on implementation
Linked-list traversal	Direct + Linear + Structural; can be Tail or Head depending on placement
Binary-tree traversal	Direct + Binary + Tree + Structural
N-Queens	Direct + Tree + Backtracking
Sudoku	Direct + Tree + Backtracking
Maze solving	Direct + Tree + Backtracking + Structural if maze is modeled recursively
Permutations	Direct + Tree + Backtracking + Combinatorial
Subsets	Direct + Tree + Backtracking + Combinatorial
Tower of Hanoi	Direct + Tree/branching + Non-tail
McCarthy 91	Direct + Nested + Non-tail
Ackermann	Direct + Nested + Multiple/branching
DFS	Direct + Structural/Graph + potentially Tree/branching
Tribonacci	Direct + Multiple + Tree + Non-tail

No.	Recursion Type	Definition / Identification	Common Examples	Typical Data / Problem
1	Direct Recursion	A function directly calls itself.	Factorial, Sum of N numbers, Power, GCD, Reverse number, Decimal-to-binary, Print 1 to N, Print N to 1	Numbers
2	Indirect Recursion	Function A calls B, and B eventually calls A.	Function A/B alternation, Even–Odd checking, State-machine recursion, Parser functions	Functions / states
3	Mutual Recursion	Two or more functions recursively call each other. It is a common form of indirect recursion.	Even–Odd, Male–Female sequence, Parser/grammar recursion, mutually dependent state processing	Multiple functions
4	Tail Recursion	The recursive call is the last operation performed.	Factorial with accumulator, Sum using accumulator, GCD, Power with accumulator, Countdown, Array traversal	Linear problems
5	Non-Tail Recursion	Some operation remains after the recursive call returns.	Factorial, Fibonacci, Reverse string, Sum of array, Tree traversal, Tower of Hanoi	Numbers / strings / trees
6	Head Recursion	Recursive call occurs before the main processing.	Print 1 to N, Reverse-order processing, Linked-list traversal, Tree traversal, String processing	Linear structures
7	Tree Recursion	Recursive calls create a branching tree of calls.	Fibonacci, Tower of Hanoi, Subset generation, Permutations, Recursive tree traversal, Catalan computation	Trees / combinatorial problems
8	Nested Recursion	A recursive call occurs inside the argument of another recursive call.	McCarthy 91, Ackermann function, Nested mathematical functions	Mathematical functions
9	Linear Recursion	Only one recursive call is made at each level.	Factorial, Sum of N numbers, Power, GCD, Linear search, Array traversal, Linked-list traversal	Linear data
10	Binary Recursion	Exactly two recursive calls are made at each applicable level.	Fibonacci, Merge Sort, Binary Tree traversal, Tower of Hanoi variants, Binary recursive search	Trees / divide-and-conquer
11	Multiple Recursion	More than two recursive calls are made at a level.	Tribonacci, Tetranacci, k-ary recursion, Multiway tree traversal, Recursive combinatorial generation	Multi-branch problems
12	Structural Recursion	Recursive calls follow the structure of the input/data structure.	Linked-list traversal, Binary tree traversal, Tree height, Tree search, Directory traversal, Expression-tree evaluation	Lists / trees / graphs
13	Backtracking Recursion	Choose → recursively explore → undo/backtrack → try another choice.	N-Queens, Sudoku, Maze solving, Rat in a Maze, Graph coloring,	Constraint problems

			Hamiltonian path, Permutations, Subsets, Combination Sum	
14	Divide-and-Conquer Recursion	Problem is divided into smaller subproblems, solved recursively, then combined/selected.	Merge Sort, Quick Sort, Binary Search, Strassen Matrix Multiplication, Closest Pair of Points, Karatsuba Multiplication	Large problems

*****THE END*****